JobPulse

Serverless Job-Market Intelligence Pipeline — Project Specification

1. What This Project Does

JobPulse is a fully serverless pipeline that continuously ingests new job postings, uses an LLM to extract structured fields from unstructured listing text, and scores each posting against a candidate's resume using an embedding-based relevance model. Postings that clear a fit threshold trigger an automated notification. The system is framed as a general-purpose job-market intelligence tool (not a single-user script), evaluated the way a production ML system would be: with a held-out labeled test set and reported precision/recall metrics on the matching model.

Core capabilities:

- Scheduled ingestion of new postings from a job board API / RSS feed
- LLM-based structured extraction: title, company, comp range, seniority, required skills, remote/visa status
- Embedding-based fit scoring between a candidate profile and each posting
- Automated alerting for high-fit postings (email/SNS)
- A persisted, queryable history of postings and scores

2. Architecture & Data Flow

Event-driven, fully serverless — no long-running servers to manage, and it scales to zero between scheduled runs.

```
EventBridge (cron) → Lambda (fetch) → S3 (raw postings) → Lambda (extract + embed) →
DynamoDB (structured records + scores) → Lambda (threshold check) → SNS/SES (alert)
```

A second, on-demand path supports querying: API Gateway → Lambda → DynamoDB, returning ranked postings to a simple frontend or CLI.

2.1 AWS Component Map

Component	AWS Service	Purpose
Scheduler	EventBridge (cron rule)	Triggers ingestion on a fixed interval (e.g. every 6h)
Ingestion	Lambda	Pulls new postings from source API, writes raw JSON to S3
Raw storage	S3	Immutable landing zone for raw postings (also your audit trail)
Extraction	Lambda + Bedrock (or OpenAI API)	LLM extracts structured fields from posting text
Embeddings	Lambda + Bedrock Titan Embeddings (or OpenAI embeddings)	Embeds posting text and candidate profile for similarity scoring
Structured store	DynamoDB	Stores structured postings, scores, and alert status

Alerting	SNS / SES	Sends notification when a posting clears the fit threshold
Query API	API Gateway + Lambda	Serves ranked/filterable results on demand
IaC	AWS CDK or Terraform	Reproducible, version-controlled infrastructure
Observability	CloudWatch Logs + Metrics + Alarms	Pipeline health, error rates, cold-start latency

GCP equivalent: Cloud Scheduler → Cloud Functions (or Cloud Run) → Cloud Storage → Cloud Functions + Vertex AI (Gemini + embeddings) → Firestore → Pub/Sub → Cloud Functions (query API), with Cloud Monitoring for observability and Terraform for IaC. The architecture maps 1:1 — pick whichever cloud better matches your target companies' stacks.

3. Match-Scoring Model (the part to make rigorous)

This is the component that turns the project from “I called an LLM” into “I built and evaluated a model.” Treat it as its own small ML task with its own dataset, baseline, and metric.

3.1 Approach

- Represent the candidate as one or more embeddings: whole-resume embedding, plus optionally per-section embeddings (skills, experience, projects) for finer-grained matching
- Embed each posting's extracted requirements/description text with the same embedding model
- Compute a base similarity score (cosine similarity between candidate and posting embeddings)
- Optionally layer a trained classifier/re-ranker on top of raw similarity, using engineered features (skill overlap count, seniority match, comp-range fit, remote/visa match) plus the embedding similarity as inputs — a gradient-boosted tree (XGBoost/LightGBM) or logistic regression works well and is easy to explain in an interview

3.2 Building an evaluation set

Manually label a sample of postings (aim for 150–300) as “good fit” or “not a fit” for your own profile/resume — this becomes your held-out test set. This is the step that unlocks a real, defensible metric instead of a vague claim.

3.3 Metrics to report

- **Precision@K** (e.g. Precision@10): of the top-10 ranked postings, what fraction are labeled good fits — this is the most interview-friendly number, directly analogous to search-ranking quality
- **Recall@K**: of all labeled good fits, what fraction appear in the top-K
- **ROC-AUC** if using a trained classifier, to show discriminative power independent of threshold
- A baseline comparison — e.g. keyword/BM25 matching only vs. embedding similarity vs. embedding + re-ranker — so you can report an improvement delta, mirroring the before/after framing used in your SQLSense project

Target resume bullet: “Built an embedding-based job-fit scoring model, evaluated on a manually-labeled set of 250 postings, achieving 88% Precision@10 (vs. 61% for keyword matching), and deployed it in a serverless pipeline that autonomously screens N postings/day.” (Replace the illustrative numbers with your actual measured results.)

4. Suggested Tech Stack

Layer	Technology
Compute	AWS Lambda (Python 3.12 runtime)
Scheduling	Amazon EventBridge
Storage (raw)	Amazon S3
Storage (structured)	Amazon DynamoDB
LLM extraction	Amazon Bedrock (Claude/Titan) or OpenAI API
Embeddings	Bedrock Titan Embeddings or OpenAI text-embedding-3
Re-ranking model	scikit-learn / XGBoost (trained offline, loaded in Lambda via a layer)
Notifications	Amazon SNS or SES
API layer	Amazon API Gateway + Lambda
IaC	AWS CDK (Python) or Terraform
CI/CD	GitHub Actions (lint, unit tests, CDK deploy)
Observability	CloudWatch Logs, Metrics, Alarms; optional X-Ray tracing

5. Advanced Features to Add

Once the core pipeline works end-to-end, these extensions add depth and give you more to talk about in interviews — pick 2–3 rather than all of them.

5.1 System-depth extensions

- **Deduplication & change detection**: hash postings to avoid reprocessing identical listings; detect and diff re-posted/edited listings
- **Multi-source ingestion**: pull from several job boards/APIs concurrently with Step Functions orchestrating parallel Lambda invocations
- **Dead-letter queues & retry logic**: SQS DLQ for failed extraction/embedding calls, with exponential backoff
- **Cost/latency optimization**: batch embedding calls, cache repeated LLM calls, compare Lambda cold-start latency against a provisioned-concurrency setup

5.2 ML-depth extensions

- **Active learning loop**: let the user thumbs-up/down recommended postings; feed labels back into periodic re-training of the re-ranker
- **Skill-gap analysis**: aggregate extracted “required skills” across postings you're a weak match for for, and surface the most common missing skills as a learning recommendation

- **Salary prediction**: train a regression model to estimate comp range when a posting omits it, using title/seniority/location/company-size features
- **Explainability**: surface *why* a posting scored high (top contributing skill-overlap terms, SHAP values from the re-ranker) rather than a black-box score

5.3 Product-polish extensions

- A lightweight dashboard (React + API Gateway) showing ranked postings, trends over time, and skill-demand analytics
- Slack/Discord bot integration as an alternative alert channel
- Weekly digest email summarizing top matches and market trends instead of per-posting alerts

6. How to Frame It on Your Resume

Present this as a general job-market intelligence tool with a pluggable candidate profile, not as a personal script — that framing reads as a reusable system rather than a one-off convenience tool, and is a better fit alongside CollabDocs (real-time systems) and SQLSense (retrieval + ML) as your third, distinct technical story: serverless/event-driven cloud infrastructure.